

ProbGen: A Browser-Based Randomized Practice Problem Generator

Mostafa Shehata, MSc. AI

Supervised by: Prof. Dr. **Michael Kohlhase**, Prof. Dr. **Florian Rabe**

Scan the **QR** code with your mobile device or use the direct link below to access the live interactive application.



Direct URL: <https://uniformal.github.io/ProbGen/>

The Learning Problem

- Students need extensive concrete numerical practice with AI/ML problems with interactive answer checking, theoretical reading alone is insufficient.

The Instructor Bottleneck

- Creating varied, mathematically sound problem instances manually is time-consuming.
- Lack of interaction with students answering and evaluation.

The Solution

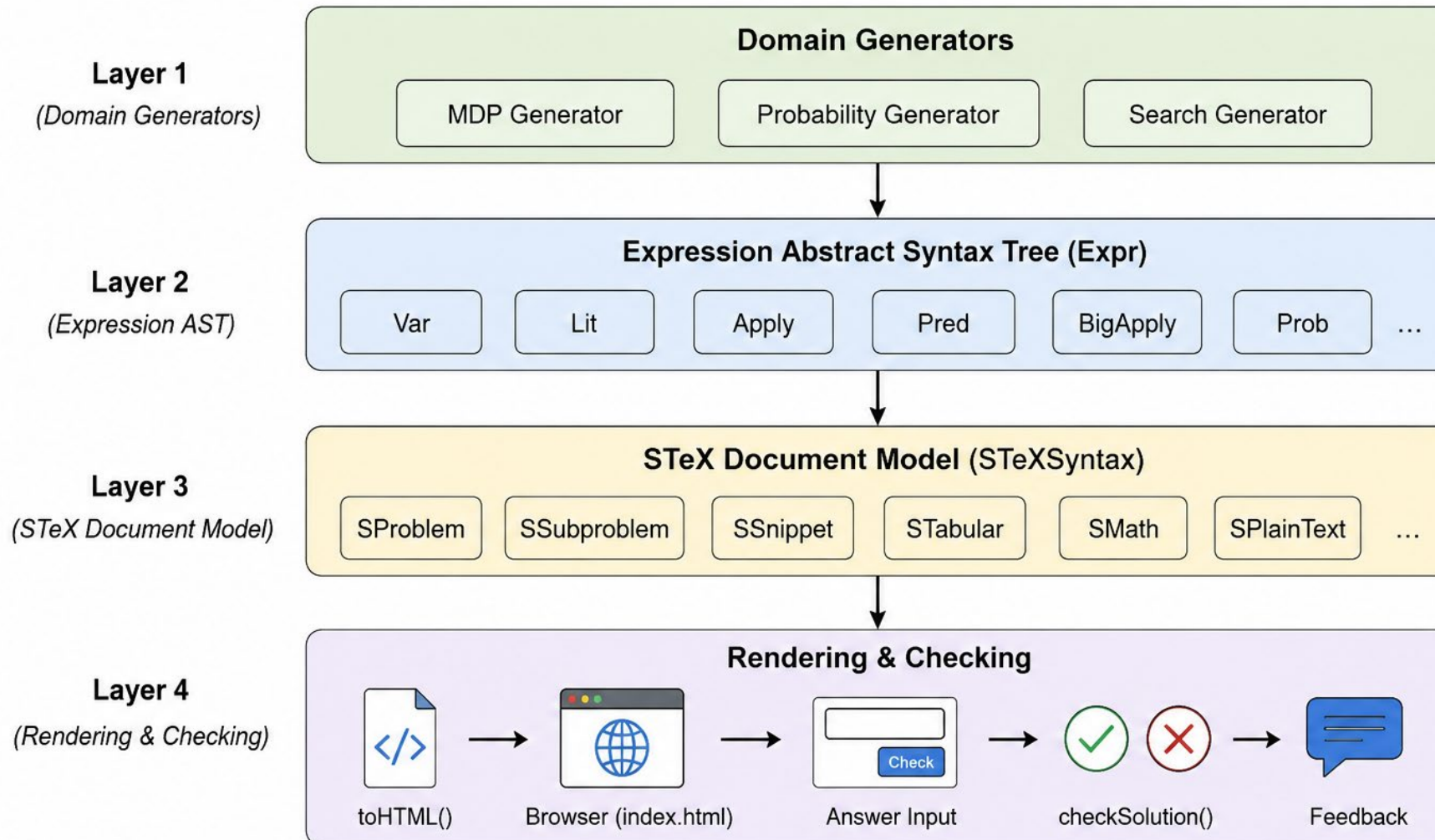
- ProbGen automatically generates unlimited random instances, solves them internally, renders them into clean HTML, and provides instant student answer verification.

- **Generate valid problem instances automatically.**
- **Support multiple AI/ML problem domains.**
- **Render generated content directly as HTML.**
- **Provide interactive answer checking and feedback.**
- **Publish the application as a static web application.**

- **Markov Decision Processes MDP**
- **Probability Problems**
- **Search Problems**
- **Adversial Search Problems**
- **CSP and Logic Problems**

System allows additional problem types to be integrated based on one line command.

System Architecture - Overview



Layer 1

Domain Generators

Sampling loops that build valid random problem instances, solving them internally before presentation.

Layer 2

Expression AST

Structured mathematical expressions supporting toSTeX, toHTML, and toText transformations.

Layer 3

STeX Document Model

Document structure wrapping mathematical snippets into formatted HTML blocks via custom string interpolators.

Layer 4

Browser Entry Point

Scala.js compiled frontend managing state mapping, user input collection, and dynamic DOM injection.

- **Implement the new problem in its problem package/folder.**
- **Test the generator and its subproblems.**
- **Register the generator in main.scala.**
- **Reuse renderProblem() and the existing STeX-to-HTML pipeline.**
- **Only extend the rendering layer if the problem requires a new visual structure.**

- **A dedicated publication branch can trigger GitHub Actions.**
- **The workflow checks out the repository and sets up Java and SBT.**
- **`sbt fastLinkJS` builds the `Scala.js` application.**
- **The generated `main.js` is copied to `./docs`.**
- **GitHub Pages uploads and deploys docs as the static site.**
- **This avoids manually rebuilding and committing `main.js` after every change.**

[2 pts] Give the probability of $\min(Y, Z) / 2 \cdot Y$ in terms of the $\{a, \dots, f\}$.

✓ Correct!

[2 pts] Give the probability of $Z \cdot Y \neq \max(0, Z)$ given $Z \neq Y$ in terms of the $\{a, \dots, f\}$.

✗ Not quite. Expected: $b + c / b + c + d + f$

[2 pts] State the Bellman Expectation Equation for a fixed policy π , i.e., for $U(\pi, s)$.

[3 pts] Consider policy π where $\pi(s) = \text{up}$. Starting with $V(s) = 0$, perform one step of policy evaluation.

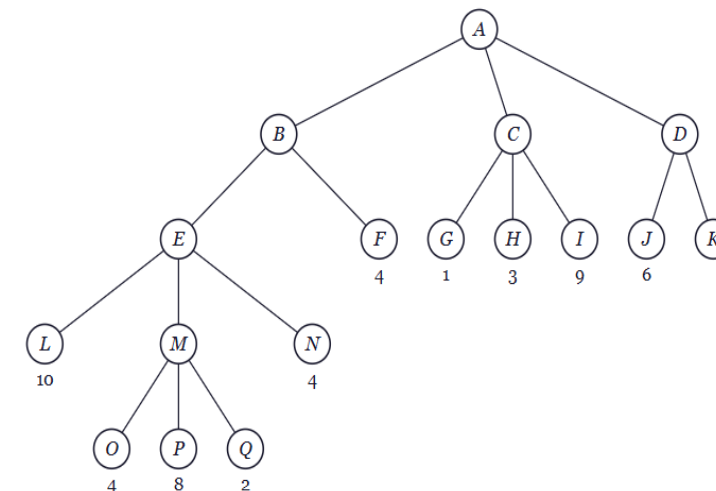
[3 pts] Suppose we solved the utilities as $U(s)$ for every state s . How do we derive the optimal policy?

[2 pts] Calculate the probability of the agent moving along to $(0, 0)$ and then to $(0, 0)$ by taking the actions **left** and **down**.

[3 pts] Now assume we use the corresponding POMDP with current belief state $b((0, 0)) = 0.5$, $b((0, 1)) = 0.5$ and apply action $a = (\text{right}, (1, 0))$. Calculate the new belief state.

Adversarial Search

Consider the following minimax game tree for the maximizing player's turn. The values at the leaves are the static evaluation function values of those states; some of those values are currently missing.



[2 pts] Give every possible label for the node K that would result in the player definitely choosing move D (no matter how ties are broken).

Original System

- Printed raw LaTeX directly to terminal
- No student input fields or answer checking
- Single developer terminal workflow

Transformed App

- Static index.html + Scala.js bundle
- Browser rendering via styled HTML spans
- Interactive answer inputs with instant feedback

Evaluation Aspect	Target Behavior	Status
Content Validity	Problem generators produce valid domain-specific instances and subproblems.	Verified
Problem Diversity	Repeated generation of new instances without page reloads.	Verified
Rendering Correctness	STeX and probability distribution tables rendered correctly as HTML.	Verified
Answer Verification	Correct, incorrect, and non-checkable states provide expected hints.	Verified

Server Independence

- ✓ **No backend required:** Zero server compute overhead for generation.
- ✓ **Local generation:** Problems built directly inside student browsers.
- ✓ **Local checking:** Answer validation executes without network requests.
- ✓ **Static hosting:** GitHub Pages serves pure static files efficiently.

Scalability & Responsiveness

- ✓ **High scalability:** Infinite independent concurrent student users.
- ✓ **Immediate response:** Rapid generation without repeated asset downloads.
- ✓ **Reproducible builds:** GitHub Actions compile reliable artifacts.
- ✓ **Version consistency:** Maintained strictly through workflow matching.

Generation & Frameworks

- ✓ **Randomised generation:** Fresh instances via reusable utilities.
- ✓ **Multi-domain integration:** MDP, Probability, Search, CSP, etc.
- ✓ **Unified abstraction:** Problem / Subproblem structure.
- ✓ **Structured math:** Expressions managed via expression/STeX models.

Execution & Verification

- ✓ **Browser-based execution:** Scala code compiled to JavaScript.
- ✓ **HTML Rendering:** Direct browser-compatible STeX structure rendering.
- ✓ **Zero external math dependencies:** Rendered using HTML & CSS.
- ✓ **Interactive Validation:** Answer fields with dedicated check features.

Limitations

- ✓ Limited number of problem types
- ✓ Basic answer normalisation
- ✓ Limited semantic checking
- ✓ No persistent student progress

Future Directions

- ✓ Expand problem domains
- ✓ Improve answer checking
- ✓ Score accumulation
- ✓ Improve and enhance UI/UX in HTML

Thank You!

Questions?